

LEA 암호모듈 기본 및 상세설계서

문서 버전: V1.2

작성일: 2026년 3월 19일

대상 모듈: KISA-LEA 소프트웨어 암호모듈

보안수준: 1

1. 암호모듈 명세

1.1 암호모듈 유형 (AS02.03)

본 암호모듈은 소프트웨어 암호모듈로 분류된다. 동적 라이브러리(.so / .dll / .dylib) 형태로 배포되며, 운영체제 위에서 실행되는 단일 파일 구성의 소프트웨어 라이브러리이다.

소프트웨어 암호모듈로 판단한 근거:

- 전용 하드웨어 없이 범용 CPU 및 운영체제 위에서 동작
- 별도 펌웨어 없이 소스코드로 구성된 순수 소프트웨어 구현체
- 동적 링크 라이브러리(.so) 형태로 배포 및 호출

운영환경:

- 운영체제: Linux (Ubuntu 20.04 LTS 64bit), Windows 10 64bit
- CPU: x86-64 (Intel/AMD)
- 컴파일러: GCC 9.4 / MSVC 2019

1.2 암호모듈 경계 및 구성요소 (AS02.07)

암호모듈의 구현물 경계(Implementation Boundary)는 liblea.so 단일 파일이며, 암호 경계(Cryptographic Boundary) 내에 포함되는 논리적/물리적 구성요소는 다음과 같다.

구성요소 목록:

파일명	역할
'lea_core.c'	LEA 블록암호 핵심 연산 (암호화/복호화)
'lea_cbc.c'	CBC 운영모드 구현
'lea_ctr.c'	CTR 운영모드 구현
'lea_gcm.c'	GCM 운영모드 구현
'lea_selftest.c'	동작 전 자가시험 및 KAT
'lea_zeroize.c'	보안 메모리 정리 구현
'lea_module.c'	모듈 초기화 및 상태 관리
'lea.h'	공개 API 헤더

암호 경계 내 모든 구성요소는 위 파일들로 한정되며, 외부 라이브러리(OpenSSL 등)는 암호 경계 외부에 위치한다.

1.3 검증대상 암호알고리즘 목록 (AS02.09)

본 암호모듈에 탑재된 검증대상 암호알고리즘 목록은 다음과 같다.

구분	알고리즘	키 길이	검증대상 여부	구현적합성 검증 방법
블록암호	LEA-128	128bit	O	KAT, MMT, MCT
블록암호	LEA-192	192bit	O	KAT, MMT, MCT
블록암호	LEA-256	256bit	O	KAT, MMT, MCT
운영모드	LEA-CBC	-	O	KAT
운영모드	LEA-CTR	-	O	KAT
운영모드	LEA-GCM	-	O	KAT

※ 각 알고리즘의 상세 보안 매개변수 명세는 별도 부록 문서를 참조한다.

1.4 비검증대상 암호알고리즘 (AS02.21)

본 암호모듈은 비검증대상 암호알고리즘을 포함하지 않는다. 모든 암호 연산은 검증대상 동작모드 내에서만 수행되며, 비검증대상 암호알고리즘과 관련이 없다.

1.5 검증대상 동작모드 실행 및 천이 (AS02.19)

본 암호모듈은 검증대상 동작모드로만 운영되며, 초기화 함수 `lea_module_init()`가 자동 호출되어 동작 전 자가시험을 수행한다. 자가시험 성공 시 검증대상 동작모드로 자동 천이된다.

동작 전 자가시험 → 성공 → 검증대상 동작모드 → 암호서비스 제공

1.6 암호모듈 명칭 및 버전 정보 (AS02.11, AS02.12)

- 모듈 명칭: KISA-LEA 소프트웨어 암호모듈
- 버전 정보: V1.2 (2026-03-19 빌드)
- 구성요소 버전: `lea_core` V1.2, `lea_cbc` V1.2, `lea_gcm` V1.2

2. 암호모듈 인터페이스

2.1 논리적 인터페이스 연관관계 (AS03.01, AS03.02)

본 암호모듈은 소프트웨어 라이브러리로서, 물리적 포트는 존재하지 않으며 논리적 인터페이스를 통해 외부와 통신한다. 논리적 인터페이스 5가지는 다음과 같이 정의된다.

인터페이스	설명
데이터 입력	함수 입력 파라미터 (평문, 암호키, IV 등)
제어 입력	API 함수 호출 자체

제어 출력	서비스 수행 중 다른 내부 서비스 호출
상태 출력	성공/실패 상태 코드 반환

2.2 서비스별 논리적 인터페이스 상세 명세 (AS03.04~AS03.08)

아래 서비스별 입출력 명세이다.

lea_cbc_encrypt():

- 데이터 입력: 평문(plaintext), 암호키(key), IV
- 제어 입력: API 함수 호출
- 상태 출력: LEA_SUCCESS(0x00), LEA_ERR_INVALID_KEY(-1), LEA_ERR_NULL_PTR(-2)

lea_cbc_decrypt():

- 데이터 입력: 암호문(ciphertext), 암호키(key), IV
- 제어 입력: API 함수 호출
- 상태 출력: LEA_SUCCESS(0x00), LEA_ERR_INVALID_KEY(-1), LEA_ERR_NULL_PTR(-2)

lea_gcm_encrypt():

- 데이터 입력: 평문, 암호키, Nonce, AAD
- 제어 입력: API 함수 호출
- 상태 출력: LEA_SUCCESS(0x00), LEA_ERR_INVALID_KEY(-1)

2.3 특정 상태 시 입출력 금지 보장 메커니즘 (AS03.07, AS03.10)

자가시험, 제로화, 오류 상태에서 데이터 출력 금지 및 제어 출력 금지를 보장하기 위해, 모든 API 함수 진입부에서 내부 상태 변수 g_module_state를 확인한다. 상태가 MODULE_STATE_OPERATIONAL이 아닌 경우 즉시 오류 코드를 반환하고 이후 연산을 수행하지 않는다.

```
if (g_module_state != MODULE_STATE_OPERATIONAL) {
    return LEA_ERR_MODULE_NOT_READY;
}
```

2.4 상태 출력 및 오류 코드 식별 체계 (AS03.11)

오류 코드는 단순한 오류와 심각한 오류로 구분된다.

코드	분류	설명
LEA_SUCCESS (0x00)	정상	성공
LEA_ERR_INVALID_KEY (-1)	단순한 오류	잘못된 키 길이
LEA_ERR_NULL_PTR (-2)	단순한 오류	NULL 포인터 입력
LEA_ERR_SELFTEST_FAIL (-100)	심각한 오류	자가시험 실패
LEA_ERR_MODULE_NOT_READY (-101)	심각한 오류	모듈 미초기화 상태

2.5 가변 길이 입력 및 데이터 형식 검증 (AS03.15)

모든 API는 입력 데이터의 길이와 포인터 유효성을 검사한다. 가변 길이 데이터에 대해 16의 배수 여부 확인, 포인터 NULL 검사를 수행하며, 검증 실패 시 LEA_ERR_INVALID_INPUT을 반환한다.

3. 역할, 서비스 및 인증

3.1 동시 운영자 지원 여부 (AS04.02)

본 암호모듈은 동시 운영자(Concurrent operator)를 지원하지 않는다. 단일 스레드 환경에서 동작하며, 멀티스레드 환경에서의 동시 호출은 호출자가 뮤텝스 등을 통해 직접 관리해야 한다.

3.2 역할 정의 (AS04.05, AS04.06)

본 암호모듈은 암호 관리자와 일반 사용자 역할을 구분한다.

- 암호 관리자(Crypto Officer): 모듈 초기화, 자가시험 실행, 보안 정리 기능을 수행
- 일반 사용자(User): 암호화, 복호화, MAC 생성, 난수 생성 서비스를 수행

3.3 서비스별 인가된 역할 매핑 (AS04.11)

서비스	인가된 역할	입력	출력
`lea_module_init()`	암호 관리자	없음	상태코드
`lea_selftest()`	암호 관리자	없음	상태코드
`lea_zeroize()`	암호 관리자	컨텍스트 포인터	상태코드
`lea_cbc_encrypt()`	일반 사용자	평문, 키, IV	암호문, 상태코드
`lea_cbc_decrypt()`	일반 사용자	암호문, 키, IV	평문, 상태코드
`lea_gcm_encrypt()`	일반 사용자	평문, 키, Nonce, AAD	암호문, Tag, 상태코드
`lea_get_version()`	일반 사용자	없음	버전 문자열
`lea_get_status()`	일반 사용자	없음	상태코드

3.4 버전 정보 출력 서비스 (AS04.13)

lea_get_version() API를 통해 암호모듈의 명칭과 버전 정보를 출력한다.

```
const char* lea_get_version(void);  
// ■■■: "KISA-LEA V1.2"
```

3.5 상태 표시 및 확인 서비스 (AS04.14)

lea_get_status() API를 통해 현재 모듈 상태를 확인할 수 있다.

```
int lea_get_status(void);  
// ■■■: MODULE_STATE_OPERATIONAL, MODULE_STATE_ERROR ■
```

3.6 사용자 호출 방식 동작 전 자가시험 (AS04.15)

사용자 동작 전 자가시험 서비스로 `lea_selftest()` API를 제공한다. 운영자가 원하는 시점에 명시적으로 호출하여 자가시험을 수행할 수 있다.

3.7 제로화 서비스 (AS04.17)

본 암호모듈은 내부적으로 메모리 정리 기능을 수행한다.

3.8 리셋/전원 종료 시 이전 인증 소멸 (AS04.43)

전원 종료 및 재부팅 시 운영체제의 로그오프 메커니즘에 의해 프로세스 메모리가 해제되며, 이전 인증 정보가 자동으로 소멸된다.

3.9 인증 데이터 보호 (AS04.44)

본 암호모듈은 자체적인 인증 메커니즘을 포함하지 않으므로, 인증 데이터를 보유하지 않는다. 운영체제 수준의 인증을 통해 암묵적으로 역할이 할당된다.

3.10 운영체제 연계 역할 할당 (AS04.56)

본 모듈은 자체 인증 메커니즘 없이 운영환경(운영체제)의 인증을 통해 암묵적 역할 할당이 이루어진다. 관리자 계정으로 로그인한 운영자는 암호 관리자 역할을, 일반 계정은 일반 사용자 역할을 수행한다.

4. 소프트웨어 보안

4.1 소프트웨어 무결성 검증 기법 (AS05.02)

소프트웨어 무결성 검증에는 HMAC-SHA256 알고리즘을 사용한다. 모듈 로드 시 동작 전 자가시험의 일환으로 HMAC-SHA256 기반 무결성 검사가 자동 수행된다.

4.2 무결성 검증키 분산 저장 (AS05.05)

HMAC-SHA256 무결성 검증키는 소프트웨어 모듈 내에 XOR 분산 저장 방식으로 하드코딩된다. 두 개의 분할 키를 런타임에 XOR 연산으로 결합하여 실제 HMAC 키를 복원한다.

```
// ■■■■ ■■■■ ■■ ■■ ■■
static const uint8_t key_part1[32] = { 0xA1, 0xB2, ... };
static const uint8_t key_part2[32] = { 0xC3, 0xD4, ... };
// ■■■■: hmac_key[i] = key_part1[i] ^ key_part2[i];
```

4.3 무결성 실패 시 오류 천이 및 임시 데이터 제로화 (AS05.06)

동작 전 자가시험 중 소프트웨어 무결성 검증에 실패할 경우 즉시 심각한 오류 상태로 천이된다. 시험 과정 중 사용된 임시 변수 및 중간 데이터는 검사 완료 후 즉시 제로화된다.

```
// ■■■■ ■■ ■■ ■
g_module_state = MODULE_STATE_ERROR;
// ■■ ■■ ■■■■
memset_s(temp_hmac_buf, sizeof(temp_hmac_buf), 0, sizeof(temp_hmac_buf));
```

```
return LEA_ERR_INTEGRITY_FAIL;
```

4.4 온디맨드 무결성 시험 API (AS05.09)

운영자 요구에 의한 무결성 시험 API로 `lea_selftest()` 함수가 제공됩니다.

4.5 안전한 배포 및 설치 전 무결성 확인 (AS05.04)

배포 시 시험기관 홈페이지에 해시값을 공시한다. 도입기관은 설치 전 해시값 비교를 통해 검증필 여부를 확인한다.

5. 운영환경

5.1 운영환경 상세 정보 (AS06.03)

항목	내용
운영체제	Linux Ubuntu 20.04 LTS (64bit), Windows 10 64bit
CPU	x86-64 (Intel Core i5 이상)
메모리	최소 512MB RAM
컴파일러	GCC 9.4 (Linux), MSVC 2019 (Windows)
컴파일 옵션	-O2 -fstack-protector-strong (Linux)

5.2 프로세스 및 가상 메모리 보호 메커니즘 (AS06.05, AS06.06)

운영체제의 가상 메모리 분리 메커니즘을 통해 각 프로세스의 메모리 공간이 독립적으로 보호된다. ASLR(Address Space Layout Randomization) 및 DEP(Data Execution Prevention) 기능을 통해 외부 프로세스의 간섭 및 인가되지 않은 접근을 방지한다.

5.3 자식 프로세스 생성 (AS06.08)

본 암호모듈은 동작 중 별도의 추가 프로세스를 생성하지 않는다. 모든 연산은 호출자 프로세스 내에서 동기적으로 수행된다.

6. 중요 보안매개변수 관리

6.1 CSP 및 PSP 보호 (AS09.01, AS09.02)

운영체제의 가상 메모리 보호 기능을 통해 암호모듈 내 CSP 및 PSP가 인가되지 않은 접근, 사용, 노출, 변경 및 대체로부터 보호된다.

6.2 난수발생기 상태 정보의 CSP 간주 (AS09.04)

난수발생기의 상태 정보(엔트로피, 시드값, V, Key), 키 생성 과정에서 발생한 중간값 등을 CSP로 간주하여 사용 후 제로화하여 관리한다.

6.3 외부 엔트로피 잡음원의 CSP 간주 (AS09.07, AS09.08)

난수발생기 시드값을 구성하기 위해 운영체제 /dev/urandom에서 수집되는 엔트로피 잡음원을 CSP로 간주한다.

잡음원	최종 엔트로피	반복 사용 횟수
/dev/urandom	256bit	1회 (재시드 기준)

6.4 SSP 생성 및 자동화된 SSP 설정 (AS09.09, AS09.10)

비밀키는 검증대상 난수발생기(HMAC-DRBG)를 이용하여 생성한다. 자동화된 SSP 설정은 본 모듈의 범위에 포함되지 않는다.

6.5 SSP 주입 및 출력 형태 (AS09.19)

소프트웨어 라이브러리 특성상 암호 경계를 넘나드는 SSP는 없다. 비밀키는 API 파라미터로 전달되며, 처리 후 내부 버퍼에서 제로화된다.

6.6 SSP 저장 및 PSP 변경 금지 (AS09.25, AS09.27)

PSP 변경 금지는 운영체제의 메모리 보호를 통해 보장된다. 전원 차단 시 키 저장 기능은 제공하지 않으며, 재시작 시 새로운 키를 생성해야 한다.

6.7 사용 후 메모리 정리 (AS09.29)

사용 후 메모리 정리를 수행한다.

7. 자가시험

7.1 자가시험 실패 시 오류 처리 및 해제 방법 (AS10.07, AS10.08)

자가시험 실패 시 심각한 오류 상태(MODULE_STATE_ERROR)로 천이된다. 오류 발생 조건은 KAT 불일치, HMAC 무결성 검증 실패이다. 오류 상태 해제를 위해서는 암호모듈 재시작(프로세스 재시작) 또는 재설치가 필요하다.

7.2 자가시험 오류 시 암호 서비스 제한 (AS10.09, AS10.10)

오류 상태에서 암호연산 금지 및 암호 서비스 제한을 보장하기 위해 모든 API 진입부에서 상태를 검사한다. 오류 상태에서는 출력 금지, 제어 출력 금지를 엄격히 적용한다.

7.3 소프트웨어 무결성 시험 전 KAT 선행 (AS10.20)

동작 전 소프트웨어 무결성 시험을 수행하기 이전에, 무결성 기술(HMAC-SHA256)에 대한 핵심기능시험(KAT)이 반드시 선행된다. 수행 순서: LEA KAT → HMAC KAT → 무결성 시험.

7.4 동작 전 핵심기능시험(KAT) 수행 방법 (AS10.24)

동작 전 핵심기능시험은 `lea_selftest()` 호출 시 자동 수행되며, 테스트벡터는 KISA 사전검증 시스템(<https://seed.kisa.or.kr>) 기준 값을 사용한다.

알고리즘	시험 항목	테스트벡터 출처
LEA-128	KAT (암호화/복호화)	KISA 사전검증 시스템
LEA-192	KAT (암호화/복호화)	KISA 사전검증 시스템
LEA-256	KAT (암호화/복호화)	KISA 사전검증 시스템
LEA-CBC	KAT	KISA 사전검증 시스템
LEA-GCM	KAT	KISA 사전검증 시스템

7.5 엔트로피 잡음원 건전성 시험 (AS10.34, AS10.25)

난수발생기 잡음원 수집 시 반복 횟수 테스트(Repetition Count Test) 및 적응성 비율 테스트(Adaptive Proportion Test)를 수행한다. 건전성 시험 실패 시 오류 상태로 천이된다.

7.6 조건부 키쌍 일치 시험 (AS10.35)

본 모듈은 공개키 기반 암호알고리즘을 포함하지 않으므로, 키쌍 일치 시험은 해당 사항 없다.

7.7 주기적 자가시험 (AS10.53)

주기적 자가시험 기능은 `lea_selftest()` API를 통해 운영자가 원하는 시점에 명시적으로 호출할 수 있다.

8. 생명주기 보증

8.1 암호모듈 동작 흐름 (AS11.08)

암호모듈의 동작 흐름은 초기화 후 서비스 제공 방식으로 진행된다. 상세한 동작 기술은 구현 코드를 참조한다.

8.2 단순 오류 발생 시 복구 (AS11.11)

잘못된 키 사이즈 입력 등 운영자 실수로 인한 단순한 오류 상태 발생 시 오류 코드를 반환하고 정상 천이하여 자동으로 정상동작 모드로 복구된다.

8.3 소프트웨어 개발환경 및 빌드 도구 (AS11.15, AS11.21)

개발 환경:

항목	내용
운영체제	Ubuntu 20.04 LTS 64bit
컴파일러	GCC 9.4.0

빌드 도구	GNU Make 4.2
컴파일 옵션	-O2 -fstack-protector-strong -D_FORTIFY_SOURCE=2
런타임 라이브러리	glibc 2.31

8.4 기능시험 및 자동화 보안 진단 (AS11.29, AS11.30)

자동화 보안 진단 도구를 이용하여 버퍼오버플로우에 취약한 함수(strcpy, strlen 등)를 점검하고 안전하게 보완하였다. 벤더 시험을 통해 기능시험 내역을 확인하였다.

8.5 수명 종료 시 안전한 소거 절차 (AS11.36)

암호모듈 사용 완료 후 안전한 소거를 위해 운영체제 명령어를 사용한다. Linux: shred -uзн 3 liblea.so, Windows: cipher /w:경로.

8.6 관리자/비관리자 안내서 (AS11.38, AS11.39)

본 암호모듈은 관리자 안내서와 비관리자 안내서를 분리하여 제공한다. 관리자 안내서에는 설치, 초기화, 자가시험, 제로화 절차를 포함하며, 비관리자 안내서에는 API 사용법 및 사용 책임을 포함한다.

9. 기타 공격 대응

9.1 타이밍 공격 대응 (AS12.02)

본 모듈은 공개키 암호알고리즘을 포함하지 않으므로 타이밍 공격 대응은 해당 사항 없다.

9.2 버퍼오버플로우 구조적 방어 (AS12.02)

모든 API 진입부에서 입력 인자 형식검증을 수행한다. 응용프로그램으로부터 전달받는 입력 인자의 크기를 검사하여 데이터 형식 및 크기 검증 후 실패 시 오류를 반환하는 구조적 방어 로직을 구현한다.

10. 보안매개변수 관리 표 (상세설계서)

분류	구분	생성	주입	출력	저장	소거
LEA 비밀키	비밀	○				○
LEA 라운드키	비밀					○
IV/카운터	공개		○			○
HMAC-DRBG 내부상태 (V, Key)	비밀	○				○

무결성 검증 HMAC 키	비밀				○(분산)	○
------------------	----	--	--	--	-------	---

본 설계서는 KCMVP 정확도 검증용 테스트 문서입니다. 의도적 위반 5개 포함: DOC-003, DOC-012, DOC-022, DOC-040, DOC-048